

Divide and conquer

CS3230 · Lecture 3

Demo course material, written for OpenPapr

Recap

- Recurrences describe recursive running times
- Today: designing recursive algorithms

The pattern

- Divide the problem into smaller parts
- Conquer each part recursively
- Combine the answers

Divide, conquer, combine

- Split the problem
- Solve the parts recursively

Combine the answers

Example: merge sort

- Split the array in half
- Sort each half
- Merge the two sorted halves

Merging

- Walk both halves with two pointers
- Take the smaller head each time
- $\Theta(n)$ for n elements in total

Merge sort, the code

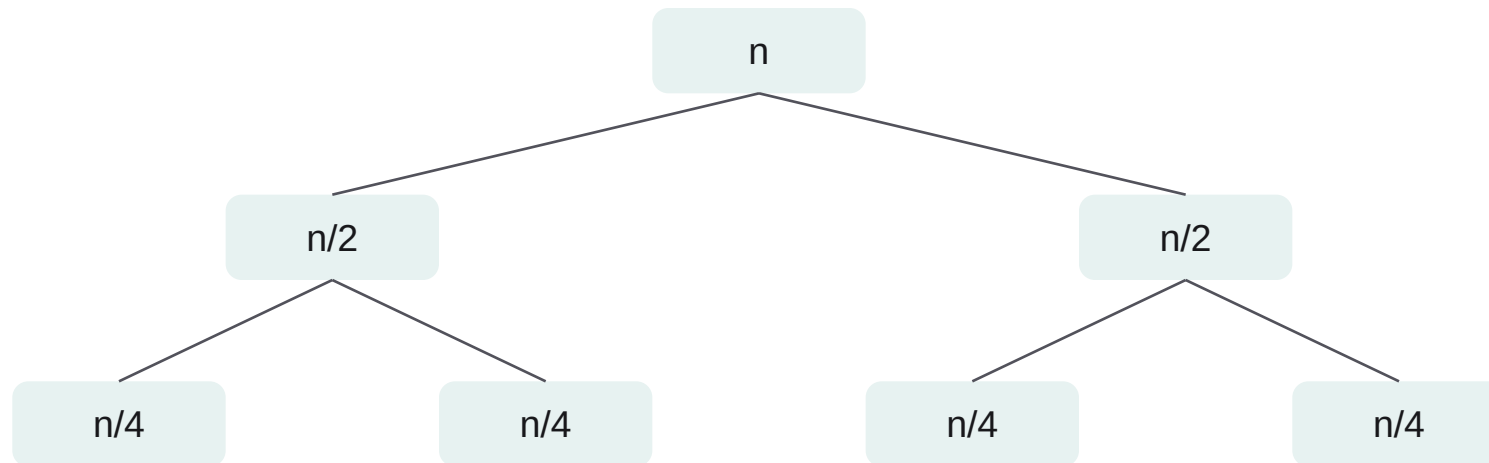
```
mergeSort(A, lo, hi):  
    mid = (lo + hi) / 2  
    mergeSort(lo, mid); mergeSort(mid+1, hi)  
    merge(lo, mid, hi)
```

Correctness

- Induction on the array length
- Base case: one element is sorted
- Merge of two sorted halves is sorted

Recursion tree

- Each level does $\Theta(n)$ work in total
- There are $\log n$ levels



Counting levels

- Sizes $n, n/2, n/4, \dots, 1$
- $\log_2 n + 1$ levels

Merge sort as a recurrence

- $T(n) = 2T(n/2) + \Theta(n)$

$$T(n) = \Theta(n \log n)$$

- Matches the recursion tree

Example: binary search

- $T(n) = T(n/2) + \Theta(1)$
- $T(n) = \Theta(\log n)$

Example: Karatsuba

- Multiply n -digit numbers with 3 half-size products
- $T(n) = 3T(n/2) + \Theta(n)$
- $\Theta(n^{1.585})$ instead of $\Theta(n^2)$

When is it worth it?

- Parts must be independent
- Combining must be cheap
- Otherwise try dynamic programming

Summary

- Divide, conquer, combine
- Write the recurrence, then solve it
- Next: the master theorem